STAT 5243 Project 2 — Interactive Data Workbench

1. Overview and Deployment

Our application is an interactive data workbench built with **Shiny for Python**. It lets users load, clean, transform, and explore tabular datasets entirely in the browser — no coding required. The interface is organized into six tabs — **Guide**, **Load**, **Overview**, **Cleaning**, **Feature Engineering**, and **EDA** — accessible from a top navigation bar styled with the Lux Bootstrap theme. The tabs are not strictly sequential: users can visit Overview and EDA at any point to inform cleaning and feature-engineering decisions. All computation runs locally; there is no REST API or external backend, so user data never leaves the machine.

Deployed application: https://019d23ea-1266-cada-1d21-45e5d97e6ea5.share.connect.posit.cloud/

To run locally: `pip install -r requirements.txt` then `shiny run app.py` and open http://127.0.0.1:8000.

The app accepts **CSV, Excel (.xlsx/.xls), JSON, and RDS** uploads and includes three built-in datasets: **Sleep, Mobile and Stress** (15,000 rows), **Iris** (150 rows), and **Tips** (244 rows).

2. Functionalities

Data Loading and Version History. Users load data in the **Load** tab by selecting a built-in dataset or uploading a file. The app displays a summary card with row count, column count, missing values, and duplicates. Every derived dataset is tracked in an in-memory **version history** with descriptive names encoding the operation (e.g., `knn_Age_01`, `log_Price_01`, `filt_age_geq_5_01`), so users can switch back to any previous version using the per-tab dataset pickers.

Overview (Decision-Support Summary). The **Overview** tab sits between Load and Cleaning, designed to inform preprocessing decisions. It shows a **missing-value table** (columns with missing counts and percentages), a **duplicate overview** (count and example rows), and a **scale review** (min, max, and mean for every numeric column).

Data Cleaning and Preprocessing. The **Cleaning** tab uses a sidebar layout with a per-tab dataset picker. Nine operations are available: handle missing values (seven strategies including **k-NN imputation** — fills missing values using k nearest neighbors from other numeric columns, with automatic feature selection and configurable k), inspect and remove duplicates, scale numeric columns (standard, min-max, robust), encode categorical columns (label, one-hot), detect and handle outliers (IQR-based), standardize text (whitespace and case), and coerce column types. The column selector auto-filters to the relevant type per operation. Users preview before applying, with before/after comparison charts.

Feature Engineering. The **Feature Engineering** tab offers **twelve** transforms: log, square, cube, interaction, ratio, binning, one-hot encoding, standardize, normalize, fill NA, drop NA, and **custom algebraic expression** (enter any pandas-eval formula like `(price - cost) / price` to create a new column). Each method includes a contextual explanation, formula display, and before/after comparison chart.

Exploratory Data Analysis. The **EDA** tab provides a per-tab dataset picker, summary tables (data preview, **separate numeric and categorical describe tables**, column types), a free-text pandas query filter, and five visualization panels — all rendered with Plotly: 1D plots (histograms/bar charts with log-scale toggles and persistent statistics), 2D plots (scatter, line, bar, box, heatmap, 2D histogram with type-labeled column selectors), regression analysis (polynomial/robust/LOWESS with Pearson r, R-squared, and p-value), multiline grouped plots, and a correlation matrix heatmap (Pearson, Spearman, Kendall).

Export. CSV download buttons on Load, Cleaning, and Feature Engineering tabs.

3. How to Use the App

1. **Load a dataset.** In the **Load** tab, select a built-in dataset or upload a file. The summary card shows row count, column count, missing values, and duplicates.
2. **Review the Overview.** The **Overview** tab shows which columns have missing values, how many duplicate rows exist, and the scale of each numeric column — helping you decide what to clean first.

3. **Clean and preprocess.** In the **Cleaning** tab sidebar, select a dataset version from the per-tab picker, choose an operation (e.g., "Handle missing values" with k-NN imputation), configure parameters, click **Preview**, then **Apply**. Each operation creates a descriptively named version in the history.
4. **Engineer features.** In the **Feature Engineering** tab, select a transform (e.g., "Custom New Column" with expression `(price - cost) / price`), preview the formula and comparison chart, then apply.
5. **Explore with EDA.** In the **EDA** tab, select a dataset version, filter with pandas query expressions, and render plots. Use the persistent statistics panel and regression R-squared to quantify relationships.
6. **Download results.** Click any **Download CSV** button to export your current dataset or preview.

4. Team Contributions

Team Member	Contribution
Cecilia Zang	Data cleaning backend (`data_cleaning.py`): 9 operations including k-NN imputation with automatic feature selection
Baixuan Chen	Feature engineering backend (`feature_engineering.py`): 12 transforms including custom algebraic expressions
Yuhan Guo	EDA backend (`eda.py`): summary functions, filtering, 6 plot families, regression analysis, and correlation matrix
Zeming Liang	Shiny UI and integration (`app.py`): 6-tab layout, Overview tab, per-tab dataset pickers, descriptive version names, Lux theme, deployment, testing, and report

5. Limitations and Future Work

- **Session-local state.** Version history is stored in server memory and lost when the app restarts.
- **Single-user design.** Concurrent users would share reactive state; production deployment would need per-session isolation.
- **k-NN performance.** k-NN imputation on very large datasets may be slow due to neighbor search; future work could add approximate nearest-neighbor algorithms.
- **Future enhancements.** Add violin and KDE density plots, support Parquet format, and add batch pipeline execution.